



**INSTITUTE FOR ADVANCED COMPUTING
AND SOFTWARE DEVELOPMENT
AKURDI, PUNE – 411044**

Documentation On
“Reservoir Water Monitoring & Analytics”
PGCP-BDA FEB 2026

Submitted By:

Group No: 13

**Rohit Jadhav (262537)
Shreepad Wadgave (262556)**

Dr. Shantanu Pathak
Project Guide

Mr. Anil Sharma
Centre Coordinator

DECLARATION

I, the undersigned hereby declare that the project report titled " Reservoir Water Monitoring & Analytics " written and submitted by me to Institute For Advanced Computing And Software Development Akurdi Pune, in the fulfilment of requirement for the award of Post Graduate Certificate Programme In Big Data Analytics (PGCP-BDA) under the guidance of Dr. Shantanu Pathak. It is my original work I have not copied any code or content from any source without proper attribution, and I have not allowed anyone else to copy my work. The project was completed using Kafka, PySpark, Hive, Airflow, Power BI and Python. The project was developed as part of my academic coursework. I also confirm that the project is original, and it has not been submitted previously for any other academic or professional purpose.

Place:

Signature:

Date:

Name: Rohit Jadhav / Shreepad Wadgave

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. Shantanu Pathak, Project Guide , for providing me with the guidance and support to complete this academic project . Their valuable insights , expertise and encouragement have been instrumental in the success of this project .

I would also like to thank my fellow classmates for their support and cooperation during the project . Their feedback and suggestions were helpful in improving the quality of the project.

I would like to extend my gratitude to Mr. Anil Sharma, Centre Coordinator , for providing me with the necessary resources and facilities to complete this project . Their support has been crucial in the timely completion of this project.

Finally , I would like to thank my family and friends for their constant encouragement and support throughout the project . Their belief in me has been a constant source of motivation and inspiration.

Thank you all for your support and guidance in completing this academic project.

ABSTRACT

The Reservoir Water Monitoring & Analytics project addresses the challenge of monitoring and analyzing large-scale reservoir water-level data collected by the Central Water Commission (CWC). Traditional spreadsheet or batch-based analysis makes it difficult to efficiently track reservoir storage, identify missing or inconsistent observations, compare current levels with previous years and derive meaningful regional or basin-level insights.

The project uses Python, Apache Kafka, Spark Structured Streaming, Apache Hive, Apache Airflow, Parquet, SQL and Power BI as the primary technologies. In addition, a Retrieval-Augmented Generation (RAG) assistant is integrated.

The scope of the project covers the complete data lifecycle from raw reservoir data ingestion to analytics and intelligent querying. The RAG component extends the system beyond conventional dashboards by providing natural-language access to reservoir information from CWC bulletins.

Table Of Contents

1. <u>INTRODUCTION</u>	1
1.1 Problem Statement	2
1.2 Scope.....	3
1.3 Aim & Objectives.....	4
2.<u>PROJECT DESCRIPTION</u>	5
2.1 Project work flow	5
2.2 Project Architecture Overview	6
2.3 Data Source & Collection	8
2.4 Streaming & Processing Frameworks Used	8
2.5 Automated Workflow & Data Storage Strategy.....	9
3.<u>SYSTEM ARCHITECTURE</u>	10
3.1 Technologies/Platforms Used & Roles	10
3.2 Apache Kafka Setup	12
3.3 Spark Structured Streaming.....	12
3.4 Data Processing & Analytical Layer.....	13
3.5 Airflow Workflow & Power BI Integration	14
3.6 RAG Assistant Architecture.....	15
4.<u>IMPLEMENTATION DETAILS</u>	16
4.1 Data Preprocessing	16
4.2 Kafka and PySpark Processing.....	16
4.3 Hive Data Warehouse.....	17
4.4 RAG Data Processing	17
4.5 Embedding and Vector Search	18
4.6 Hybrid RAG Retrieval.....	18

4.7 LLM Integration.....	19
4.8 RAG Assistant Interface.....	19
5. <u>PROJECT REQUIREMENTS</u>	20
5.1 Hardware Requirements.....	20
5.2 Software & Libraries Used.....	21
6. <u>FUTURE SCOPE</u>	23
7. <u>CONCLUSION</u>	25
8. <u>REFERENCES</u>	27

Table of Figure:

Figure 1 <i>Project work flow diagram 1</i>	5
Figure 2 <i>Project work flow diagram 2</i>	6
Figure 3 <i>Screenshot 1</i>	26
Figure 4 <i>Screenshot 2</i>	26

CHAPTER 1

INTRODUCTION

Reservoirs are critical for water supply, irrigation, hydropower generation and management of water resources. The Central Water Commission (CWC) publishes reservoir-level information containing parameters such as reservoir name, basin, location, date, full reservoir level, live capacity, storage and water level. The availability of multi-year reservoir data provides an opportunity to analyze storage patterns, compare reservoir conditions across years and identify changes in water availability.

The Reservoir Water Monitoring & Analytics project develops an end-to-end big data pipeline for processing and analyzing CWC reservoir data. Historical reservoir datasets from 2022 to 2025 are ingested through Apache Kafka, processed using Apache Spark Structured Streaming, and stored in analytical formats using Hive and Parquet. Apache Airflow is used to orchestrate the data-processing workflow, while Power BI provides dashboards for visualization and analysis.

The project also incorporates a Retrieval-Augmented Generation (RAG) Assistant to provide natural-language access to CWC reservoir bulletins. The RAG system combines structured database lookup with semantic search using SQLite, Sentence Transformers and FAISS, followed by response generation using a large language model. A Flask-based web interface allows users to ask questions about reservoir information and receive answers grounded in the available CWC data.

1.1 Problem Statement

India tracks 150+ major reservoirs across 21 river basins. The Central Water Commission (CWC) publishes this data in two disconnected forms. One is Historical daily records as raw CSV exports (2022–2025) with no cleaning, dedup or query layer. Other is Weekly “Live Storage Status” bulletins released only as dense, table-heavy PDFs unreadable by any downstream system without manual effort. There is no automated pipeline to clean & warehouse the historical data and no way for a non-technical user to simply ask a question and get an instant, trustworthy answer from the latest bulletin.

The problem can be summarized as follows:

- **Large-volume data processing:** Multi-year CWC datasets contain a large number of reservoir observations that require systematic processing and transformation.
- **Data quality issues:** The datasets contain missing values, duplicate observations, inconsistent records, and fields such as subbasin with incomplete availability.
- **Difficult historical comparison:** Comparing reservoir storage and water levels across different years and dates is difficult using raw CSV files alone.
- **Limited analytical accessibility:** Raw datasets do not provide an easy way for users to obtain reservoir-level, basin-level or regional insights.
- **Limited natural-language access:** Conventional dashboards require users to select filters and interpret visualizations, whereas users may also need to ask direct questions about reservoir conditions.

1.2 Scope

The project focuses on data engineering, analytics, visualization and information retrieval. The scope of the project covers the complete processing lifecycle of CWC reservoir data:

- **Collection, Organization & Data Engineering:** Collection and organization of multi-year CWC reservoir datasets from 2022–2025, along with data cleaning and transformation using PySpark, including date standardization, duplicate handling, field standardization & data validation.
- **Streaming, Storage & Workflow Automation:** Streaming-based data ingestion using Apache Kafka and processing using Spark Structured Streaming, with storage and querying using Hive and Parquet. Workflow automation and pipeline orchestration using Apache Airflow for efficient and reliable data processing.
- **Analytics, Visualization & Monitoring:** Development of Power BI dashboards for reservoir storage analysis, water-level trends, year-wise comparisons and continuous monitoring, providing structured visual insights into reservoir conditions and historical patterns.
- **RAG System & User Interface:** Development of a hybrid RAG system using SQLite for exact structured retrieval and FAISS-based semantic search for unstructured bulletin information. Development of a Flask-based web interface through which users can ask natural-language questions and retrieve relevant reservoir information.

1.3 Aim & Objective

To develop a scalable Reservoir Water Monitoring & Analytics system that integrates big-data technologies, automated data processing, visualization and RAG-based natural-language querying to support efficient analysis of CWC reservoir data.

- **Implement Scalable Data Ingestion and Processing:** Use Apache Kafka for data ingestion and streaming, with Spark Structured Streaming/PySpark for scalable data cleaning, transformation, validation and processing of reservoir data.
- **Organize and Store Analytical Data:** Structure the processed reservoir data into analytical tables suitable for querying, reporting and downstream analysis, using Hive and Parquet for efficient storage and data retrieval.
- **Automate the Data Engineering Pipeline:** Use Apache Airflow to automate pipeline execution, scheduling, task dependencies and workflow management across the reservoir data processing stages.
- **Develop Reservoir Analytics and Dashboards:** Create Power BI dashboards for reservoir-level analysis, storage monitoring, historical trends, year-wise comparisons and visualization of key reservoir indicators.
- **Implement a Hybrid RAG Assistant:** Develop a hybrid RAG Assistant capable of retrieving exact reservoir information and relevant CWC weekly bulletin content using SQLite for structured retrieval and FAISS with Sentence Transformers for semantic search, combined with a large language model for grounded natural-language response generation.

CHAPTER 2

PROJECT DESCRIPTION

This The Reservoir Water Monitoring & Analytics project is an end-to-end big-data system designed to process, transform, store and analyze multi-year reservoir data published by the Central Water Commission (CWC). The project combines batch historical data with a streaming-oriented processing architecture to demonstrate scalable data ingestion and processing. The processed data is made available for analytical querying and Power BI visualization, while a separate RAG-based assistant provides natural-language access to CWC reservoir bulletins.

2.1 Project Work Flow

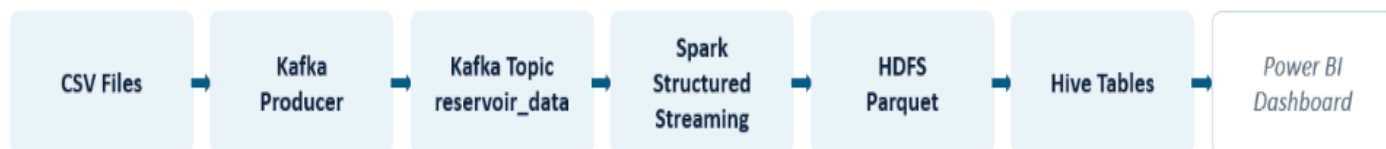


Fig 1: Project work flow diagram 1

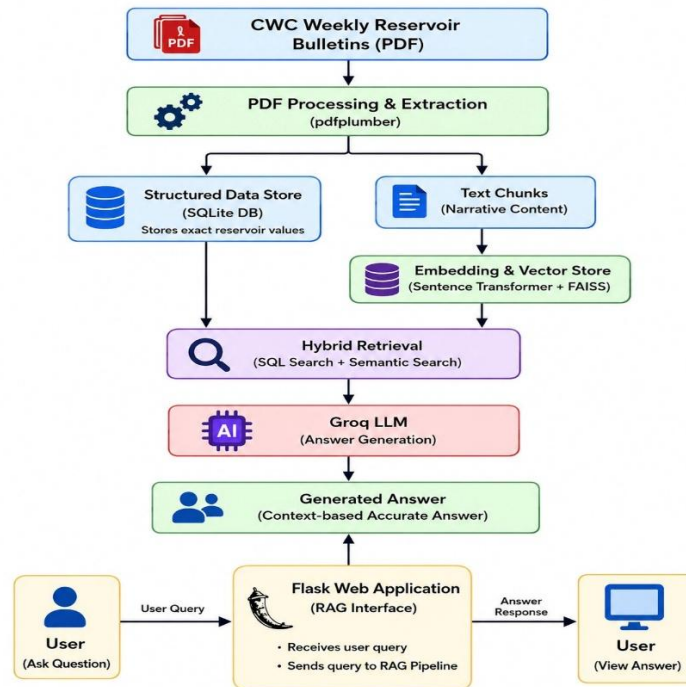


Fig 2: Project work flow diagram 2

2.2 Project Architecture Overview

The architecture follows a distributed, data-processing and AI-based information retrieval model with the following major components:

- **Python Kafka Producer:** Publishes reservoir records to Kafka for continuous data ingestion.
- **Apache Kafka:** Acts as the streaming ingestion layer, decoupling data production from downstream processing.

- **Apache Spark Structured Streaming:** Consumes Kafka records and performs schema handling, date standardization, duplicate management, data-quality processing and other transformations.
- **Apache Hive & Parquet:** Organize and store the processed datasets for analytical and downstream use.
- **Apache Airflow:** Provides workflow orchestration by managing task execution, dependencies and scheduling across ingestion, processing, storage and validation stages.
- **Power BI:** Uses the processed analytical data to create dashboards for reservoir storage, water-level trends, year-wise comparisons and other monitoring metrics.
- **CWC Weekly Bulletin Ingestion Pipeline:** Processes weekly Central Water Commission (CWC) bulletin PDFs to extract and prepare textual knowledge for retrieval.
- **SQLite:** Stores exact and structured reservoir records, enabling direct lookups for specific reservoir-related queries.
- **FAISS Vector Database:** Stores embeddings generated from bulletin text and enables semantic retrieval of relevant information.
- **LLM:** Receives the retrieved structured and semantic context to generate natural-language responses to user queries.
- **Flask Web Interface:** Provides the user-facing application through which users submit questions and receive generated answers.

2.3 Data Source & Collection

- **Source:** Central Water Commission (CWC) reservoir-level CSV datasets for 2022, 2023, 2024 and 2025, containing historical reservoir monitoring data.
- **Data Size:** 163,801 reservoir records across four years.
- **Year-wise Distribution:**
 - 2022: 43,435 records.
 - 2023: 42,458 records.
 - 2024: 51,576 records.
 - 2025: 26,332 records.
- **Attributes:**
 - Reservoir Information: Reservoir_name, Basin, subbasin, Agency_name.
 - Location: Lat, Long.
 - Time: Date, Year, Month.
 - Reservoir Measurements: Full_reservoir_level, Live_capacity_FRL, Storage, Level.
- **RAG Data Source:** CWC weekly reservoir bulletins in PDF format are used as the knowledge source for the RAG Assistant.

2.4 Streaming & Processing Frameworks Used

- **Apache Kafka:**
 - **Producer:** A Python-based producer reads reservoir records from the CSV datasets and publishes them to the configured Kafka topic as a continuous data stream.
 - **Streaming Layer:** Kafka acts as the data ingestion and buffering layer, decoupling data production from downstream Spark processing.

- **Spark Structured Streaming:**

- **Data Processing:** Consumes reservoir records from Kafka using Spark Structured Streaming.
- **Schema & Type Handling:** Applies the required schema and performs data-type conversions.
- **Data Cleaning:** Standardizes dates and reservoir/categorical fields, identifies missing values and handles duplicate records.
- **Transformation:** Creates derived fields required for analysis and organizes reservoir observations for downstream use.
- **Analytical Preparation:** Prepares the processed datasets for storage in Hive and Parquet.

2.5 Automated Workflow & Data Storage Strategy

- **Apache Airflow:**

- Automates and orchestrates the pipeline stages.
- Manages task execution order and dependencies between ingestion, processing, storage and validation tasks.

- **Data Ingestion:**

- Raw CSV Files: Store the original CWC reservoir datasets.
- Kafka Topic: Acts as the streaming transport and ingestion layer.

- **Processed Data Storage:**

- Parquet: Stores transformed datasets in an efficient analytical format.
- Hive Tables/Views: Provide SQL-based access to the processed analytical data.
- Power BI: Connects to the analytical data for dashboards and reporting.

- **RAG Data Storage:**

- SQLite Database: Stores exact structured reservoir records for direct lookup.
- chunks.json: Store extracted bulletin text and associated metadata.
- FAISS Index: Stores vector embeddings of bulletin content for semantic retrieval.

- **RAG Processing Workflow:**

- PDF Ingestion: Extracts structured reservoir information and unstructured bulletin text, generating the SQLite database and text chunks.
- Indexing: Converts bulletin text into embeddings and creates the FAISS vector index.
- Retrieval: Uses SQLite for exact reservoir queries and FAISS for semantic bulletin retrieval.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 Technologies/Platforms Used & Roles

- **Ubuntu VM**
 - Provides the development and execution environment for the big-data processing components.
 - Hosts services such as Kafka, Spark, Hive and Airflow.
- **Apache Kafka**
 - Acts as the streaming ingestion layer for reservoir records.
 - Receives data from the Python-based Kafka Producer through a dedicated reservoir topic.
- **Apache Spark**
 - Performs distributed stream processing and data transformation.
 - Consumes reservoir records from Kafka using Structured Streaming.
- **Apache Hive**
 - Provides SQL-based access to processed reservoir datasets.
 - Maintains analytical tables and views for downstream analysis.
- **Apache Airflow**
 - Automates and orchestrates the end-to-end data pipeline.
 - Manages task dependencies, execution order and validation.
- **Parquet**
 - Stores processed reservoir data in a columnar and analytics-friendly format.
 - Provides efficient storage and retrieval for large datasets.
- **Power BI**
 - Consumes processed analytical data for visualization.

- Provides dashboards for reservoir storage, water-level trends and year-wise comparisons.
- **RAG Technologies**
 - **SQLite:** Stores structured reservoir records for exact lookup.
 - **Sentence Transformers:** Generates embeddings from CWC bulletin text.
 - **FAISS:** Provides vector indexing and semantic retrieval.
 - **Groq/Llama:** Generates natural-language responses using retrieved context.
 - **Flask:** Provides the web interface for user interaction with the RAG Assistant.

3.2 Apache Kafka Setup

- **Kafka Producer**
 - Reads reservoir records from the CWC CSV datasets.
 - Publishes the records to the configured Kafka topic.
- **Reservoir Kafka Topic**
 - Acts as the communication channel between the producer and Spark Streaming application.
 - Decouples data ingestion from downstream processing.
- **Producer Configuration**
 - Configures the Kafka broker, topic, serialization and message delivery settings.
 - Streams reservoir records continuously for downstream consumption.

3.3 Spark Structured Streaming

- **Kafka Consumer**
 - Connects to the reservoir Kafka topic and consumes incoming records.
 - Reads the data using Spark Structured Streaming.
- **Micro-batch Processing**

- Processes incoming reservoir records in micro-batches.
- Enables continuous processing while maintaining scalable execution.
- **Data Cleaning & Transformation**
 - Applies schema and data-type conversion.
 - Standardizes dates and reservoir fields.
 - Handles duplicates and missing values.
 - Creates required derived fields for analysis.
- **Validation**
 - Performs data-quality checks on processed reservoir observations.
 - Ensures that the output conforms to the required analytical structure.
- **Output to Hive/Parquet**
 - Writes processed data to Parquet for efficient storage.
 - Makes the processed datasets available through Hive tables and views.

3.4 Data Processing & Analytical Layer

- **Cleaning & Duplicate Handling**
 - Standardizes reservoir and categorical fields.
 - Identifies and handles duplicate reservoir-date observations and missing measurements.
- **Reservoir Dimension Table**
 - Maintains relatively stable reservoir-level information such as reservoir name, basin, agency and location.
 - Provides the reference structure for analytical processing.
- **Daily Reservoir Fact Table**
 - Stores time-based reservoir observations and measurements.
 - Includes fields such as date, storage, level and other monitoring attributes.

- **Hive Tables/Views**
 - Provide SQL-based access to the processed analytical datasets.
 - Support querying, analysis and reporting.
- **Parquet Storage**
 - Stores transformed data in a columnar format.
 - Enables efficient analytical queries and scalable storage.

3.5 Airflow Workflow & Power BI Integration

- **Airflow DAG/Workflow**
 - Defines and automates the sequence of pipeline operations.
 - Coordinates ingestion, processing, storage and validation stages.
- **Task Dependencies**
 - Ensures that downstream tasks execute only after the required upstream tasks are completed.
 - Maintains an organized and repeatable processing workflow.
- **Pipeline Validation**
 - Validates processed datasets before making them available for analytical use.
 - Helps identify processing or data-quality issues.
- **Power BI Integration**
 - Consumes the processed analytical data from the Hive/Parquet layer.
 - Converts reservoir data into interactive dashboards and reports.
- **Dashboard/Analytical Use Cases**
 - Reservoir storage monitoring.
 - Water-level trend analysis.
 - Year-wise reservoir comparisons.
 - Reservoir and basin-level monitoring metrics.

3.6 RAG Assistant Architecture

- **CWC PDF Ingestion**
 - Processes weekly Central Water Commission (CWC) reservoir bulletins in PDF format.
 - Extracts structured reservoir information and unstructured bulletin text.
- **SQLite Structured Storage**
 - Stores extracted structured reservoir records.
 - Supports exact lookups for named reservoirs and specific structured queries.
- **Text Chunk Extraction**
 - Divides bulletin text into smaller chunks suitable for semantic retrieval.
 - Stores the extracted text and associated metadata.
- **Sentence Transformer Embeddings**
 - Converts bulletin text chunks into numerical vector representations.
 - Captures the semantic meaning of the bulletin content.
- **FAISS Vector Index**
 - Indexes the generated embeddings for efficient similarity search.
 - Retrieves bulletin content that is semantically relevant to a user's query.
- **Hybrid Retrieval**
 - Uses SQLite for exact structured reservoir lookups.
 - Uses FAISS for semantic retrieval from bulletin content.
 - Combines the retrieved information as context for answer generation.
- **Groq/Llama Generation**
 - Receives the retrieved context and user query.
 - Generates a natural-language response using the available reservoir information.

CHAPTER 4

IMPLEMENTATION DETAILS

4.1 Data Preprocessing

- **Data Collection**
 - Collected reservoir data from CWC weekly bulletins and historical reservoir datasets.
 - Combined the available datasets for downstream processing.
- **Data Cleaning**
 - Identified and handled missing, invalid and duplicate records.
 - Standardized reservoir names and categorical fields where required.
- **Data Transformation**
 - Converted date fields into standardized date formats.
 - Converted numerical attributes into appropriate data types.
 - Prepared the datasets for Kafka, PySpark, Hive and RAG processing.

4.2 Kafka and PySpark Processing

- **Kafka Producer**
 - Reads reservoir records from the prepared datasets.
 - Publishes records to the `reservoir_data` Kafka topic.
- **PySpark Structured Streaming**
 - Consumes reservoir records from the Kafka topic.
 - Processes the incoming data using micro-batch streaming.
- **Data Processing**

- Applies schema validation, data cleaning and required transformations.
- Handles duplicates, missing values and data-type inconsistencies.
- **Parquet Storage**
 - Stores the processed reservoir data in Parquet format.
 - Provides an efficient columnar format for analytical processing.

4.3 Hive Data Warehouse

- **Hive Database**
 - Created the `reservoir_db` Hive database for analytical storage.
- **Fact and Dimension Tables**
 - Implemented fact and dimension tables to organize reservoir information.
 - Supports structured analysis of reservoir observations.
- **Stored Information**
 - Includes reservoir storage, water level, capacity, basin and date-related information.
- **Analytical Metrics**
 - Generated metrics such as storage percentage and change in storage.
 - Enables comparison of reservoir conditions across dates and years.

4.4 RAG Data Processing

- **PDF Extraction**
 - Extracted text and reservoir information from CWC weekly PDF bulletins using `pdfplumber`.
- **Structured Data**
 - Stored numerical and structured reservoir information in SQLite.
 - Enables exact retrieval of reservoir-specific information.

- **Text Processing**

- Divided extracted bulletin text into smaller chunks.
- Prepared the chunks for semantic retrieval and embedding generation.

4.5 Embedding and Vector Search

- **Text Embeddings**

- Generated vector embeddings for bulletin text using Sentence Transformers.
- Converts textual information into numerical representations for similarity search.

- **FAISS Vector Index**

- Stored the generated embeddings in a FAISS index.
- Enables efficient similarity-based retrieval of relevant bulletin content.

- **Semantic Search**

- Retrieves the most relevant CWC bulletin information based on the user's query.

4.6 Hybrid RAG Retrieval

- **SQLite Retrieval**

- Performs exact structured lookups for numerical and reservoir-specific information.

- **FAISS Retrieval**

- Performs semantic searches for relevant descriptive and narrative bulletin content.

- **Hybrid Approach**

- Combines structured SQLite retrieval with FAISS semantic retrieval.
- Improves the accuracy of both numerical and descriptive question answering.

- **Context Preparation**

- Combines the retrieved information into relevant context.
- Passes the context and user query to the LLM for response generation.

4.7 LLM Integration

- **Groq API**

- Integrated the Groq API for natural-language answer generation.

- **Context-Based Generation**

- Provides the retrieved context and user's question to the LLM.
- Generates responses based on the available reservoir information.

- **Response Accuracy**

- Preserves exact numerical values such as BCM measurements and percentages.
- Reduces unsupported responses by restricting generation to the retrieved reservoir information.

4.8 RAG Assistant Interface

- **Web Interface**

- Developed a web-based interface for users to submit reservoir-related questions.

- **RAG Integration**

- Connects user queries to the hybrid retrieval and LLM generation pipeline.
- Automatically retrieves relevant structured and unstructured information.

- **Response Display**

- Presents the generated answers in a clear and easy-to-understand format.

CHAPTER 5

PROJECT REQUIREMENT

5.1 Hardware Requirements

- **Ubuntu Virtual Machine:**
 - Hosts the complete big-data environment including Hadoop, Kafka, Spark, Hive and Airflow.
 - Recommended allocation: **4 CPU cores, 8 GB RAM and 50 GB storage.**
- **Processor:**
 - Minimum: **2 CPU cores.**
 - Recommended: **4 or more CPU cores** for concurrent execution of Kafka, Spark, Hive and Airflow.
- **RAM:**
 - Minimum: **8 GB RAM.**
 - Recommended: **12–16 GB RAM** when running Hadoop, Kafka, Spark, Hive, Airflow and RAG components simultaneously.
- **Storage:**
 - Minimum: **50 GB free disk space.**
 - Recommended: **100 GB or more** for raw CSV files, processed Parquet datasets, HDFS data, Hive tables, Kafka logs, Airflow files, CWC PDF bulletins and RAG indexes.
- **Network Connectivity:**
 - Stable Internet connectivity is required for Python package installation and communication with the Groq API.

5.2 Software & Libraries Used

- **Operating System – Ubuntu:**

- Provides the primary execution environment for the big-data pipeline.
- Hosts Kafka, Spark, Hadoop, Hive and Airflow services.

- **Java 8 (OpenJDK 1.8.0_312):**

- Required runtime for Hadoop, Spark and other Java-based big-data components.

- **Python 3.7:**

- Used for data processing, Kafka producer scripts, PySpark jobs and Airflow workflows.
- Python 3.7 is used for the Airflow environment.

- **Hadoop 2.7.3:**

- Provides the distributed storage and big-data ecosystem foundation.
- HDFS is used for storing datasets and intermediate processing files.

- **Apache Spark 2.4.5:**

- Used for distributed data processing and Structured Streaming.
- Consumes reservoir records from Kafka and performs cleaning, transformation and validation.

- **Apache Kafka 2.2.1:**

- Provides the real-time streaming ingestion layer.
 - The Python producer publishes reservoir observations to the configured Kafka topic.
- **Apache Hive 2.3.6:**
 - Provides SQL-based querying and analytical storage.
 - Used to create reservoir fact/dimension tables and analytical views.
- **Apache Airflow 1.10.10:**
 - Automates the data-processing workflow.
 - Manages dependencies between ingestion, processing, storage and validation tasks.

CHAPTER 6

FUTURE SCOPE

1. Reservoir Level Forecasting

- Machine learning or time-series models can be integrated to predict future reservoir storage and water levels.
- Historical observations can be used to identify seasonal patterns and estimate future reservoir conditions.

2. Anomaly Detection

- Automated anomaly detection can identify unusual changes in storage or water levels.
- Sudden increases or decreases can be flagged for further investigation.

3. Scalable Cloud Deployment

- The complete pipeline can be migrated to cloud infrastructure for larger datasets and distributed processing.
- Cloud-based storage, compute and managed streaming services can be introduced when the volume and frequency of incoming data increase.

4. Automated CWC Data Collection

- Replace manual CSV placement with an automated mechanism to periodically collect the latest CWC reservoir data and bulletins.
- Newly available data can be automatically introduced into the Kafka ingestion pipeline.

Comparison of technologies with cloud platform alternatives:

Project Technology	Technology Used	AWS Alternative	Azure Alternative	GCP Alternative
Data Storage	Hadoop HDFS / Parquet	Amazon S3	Azure Data Lake Storage	Google Cloud Storage
Streaming	Apache Kafka	Amazon MSK / Kinesis	Azure Event Hubs	Pub/Sub
Data Processing	Apache Spark	Amazon EMR	Azure Databricks	Dataproc
SQL/Data Warehouse	Apache Hive	Amazon Athena / Glue Data Catalog	Azure Synapse Analytics	BigQuery
Workflow Orchestration	Apache Airflow	Amazon MWAA	Managed Airflow	Cloud Composer
Visualization	Power BI	Power BI / QuickSight	Power BI	Looker / Power BI
RAG Structured Storage	SQLite	Amazon RDS / DynamoDB	Azure Database/Cosmos DB	Cloud SQL/ Firestore
RAG Vector Search	FAISS	Amazon OpenSearch	Azure AI Search	Vertex AI Vector Search
LLM	Groq/Llama	Amazon Bedrock	Azure OpenAI Service	Vertex AI / Gemini

Summary Table based on Advantages & Disadvantages:

Technology / Approach	Advantages	Disadvantages
Ubuntu VM + Open-Source Stack	Low cost, full control, good for learning	Limited scalability, manual maintenance
AWS	Highly scalable, managed services, production-ready	Usage-based cost, more configuration
Azure	Excellent Power BI integration, scalable, managed services	Can become costly, Azure-specific knowledge required
GCP	Strong analytics & AI capabilities, scalable	Usage-based cost, requires GCP knowledge

CHAPTER 7

CONCLUSION

The Reservoir Water Monitoring & Analytics project provides an end-to-end framework for processing and analyzing multi-year Central Water Commission (CWC) reservoir data. Apache Kafka and Spark Structured Streaming enable scalable data ingestion and processing, while Hive and Parquet provide efficient analytical storage.

Apache Airflow automates the workflow, and Power BI provides dashboards for reservoir storage, water-level trends, year-wise comparisons and data-quality analysis.

The integrated RAG-based Reservoir Assistant enables natural-language queries over CWC bulletins. It combines SQLite for exact structured retrieval, Sentence Transformers and FAISS for semantic search, and an LLM for response generation. Overall, the project combines big-data processing, workflow automation, business intelligence and intelligent information retrieval into a unified reservoir monitoring solution, with scope for future enhancements such as real-time monitoring, forecasting and anomaly detection.

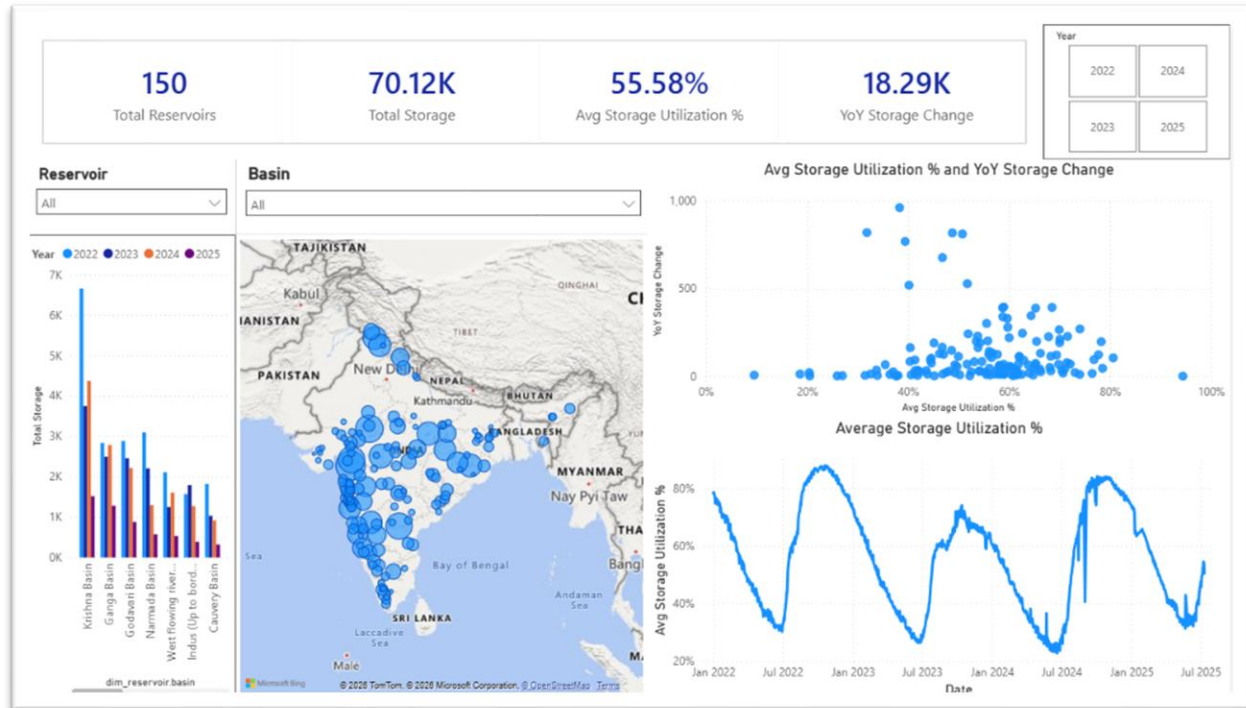


Fig 3 : Screenshot 1

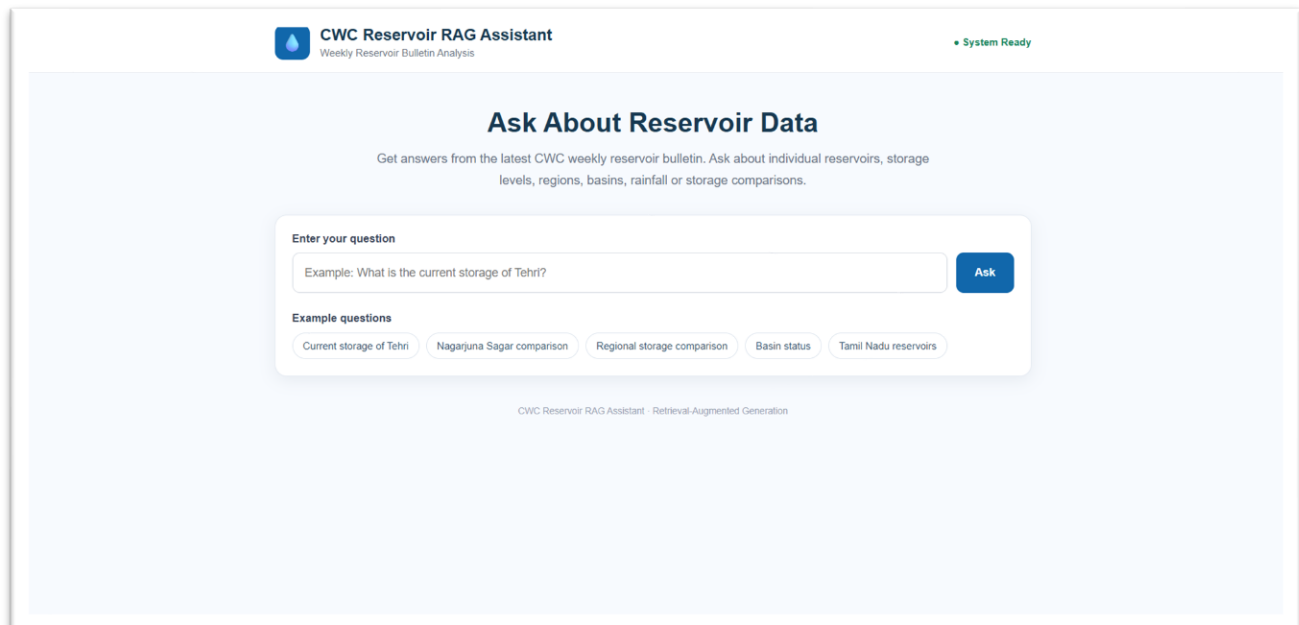


Fig 4 : Screenshot 2

CHAPTER 8

REFERENCES

1. Central Water Commission (CWC). Reservoir Storage Bulletin and Water Resources Information.
<https://cwc.gov.in/>
2. Apache Kafka Documentation.
<https://kafka.apache.org/documentation/>
3. Apache Spark Streaming Guide.
<https://spark.apache.org/docs/latest/streaming-programming-guide.html>
4. Apache Hadoop Documentation.
<https://hadoop.apache.org/docs/r2.7.3/>
5. Apache Hive Documentation.
<https://hive.apache.org/>
6. Facebook AI Research. FAISS – A Library for Efficient Similarity Search and Clustering of Dense Vectors.
<https://faiss.ai/>
7. Microsoft Power BI Documentation.
<https://learn.microsoft.com/power-bi/>
8. Groq API Documentation.
<https://console.groq.com/docs/>